



VIETNAM ACADEMY OF SCIENCE AND TECHNOLOGY
UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI

Requirement Analysis

Software Engineering

KIEU Quoc Viet HUYNH Vinh Nam

ICT Department, USTH



Agenda

- 1 Review
- 2 Requirement Analysis
- 3 WRPSM Model

1. Review

Review

Requirements Analysis

What are we building?

Functional & non-functional specs.

Design

How are we going to build it?

System architecture & component design.

Development (Implementation)

Let's build it!

Writing code based on the design.

Testing (Verification)

Does it work correctly?

Testing & validating the system.

Deployment

Ship it to the world!

Releasing the product to production.

Maintenance

Keep it alive!

Updates, patches & ongoing support.

2. Requirement Analysis

Requirements Definition

Requirements = **exact specifications of what the software should do**
(not *how* to build it — that is design)

Requirements Definition

Requirements = **exact specifications of what the software should do**
(not *how* to build it — that is design)

Requirement

"Capture the user's email address and a comment"

→ This is the **what** (requirement)

Not a Requirement

"Use CSS and HTML to build the front end"

→ This is the **how** (design)

Captures the **what**, not the **how**

Finds the **goals** of the system: *what do we want it to accomplish?*

Usually produces a **requirements document** — bullet lists or formal specs

Why Requirements Matter

Golden Rule: Spending time strategically up front reduces cost and time later on.

Upfront time spent	Cost overrun	Example (200h project)
0%	200%	600h total
3%	100%	400h total
5%	50%	300h total
10–20%	40%	280h total (golden zone)

Key insight: Just **10 hours** of upfront planning on a 200h project saves **300 hours** of backend problems.

Cost overrun = wasted **time** + wasted **money** (longer duration → more engineer costs → delayed revenue).

Requirements vs. Specifications

User Requirements

Written in **plain language** —
no tech terms

Anyone can read and understand it

Written from the **client's point of view**

System Specifications

Written for **developers** —
more detail allowed

Breaks each requirement into
smaller conditions

Still **no technology** chosen
yet (no MySQL, React...)

Requirement → what the *client* wants

Specification → developer notes to meet that requirement

Requirements vs. Specifications — Examples

Example 1: Online Shopping

User Requirement

The user should be able to place an order and pay online.

Requirements vs. Specifications — Examples

Example 1: Online Shopping

User Requirement

The user should be able to place an order and pay online.

System Specifications

Accept credit card, debit card, and e-wallet payments

Send order confirmation to user's email after payment

Store order history for at least 12 months

Requirements vs. Specifications — Examples

Example 1: Online Shopping

User Requirement

The user should be able to place an order and pay online.

System Specifications

Accept credit card, debit card, and e-wallet payments

Send order confirmation to user's email after payment

Store order history for at least 12 months

Example 2: User Login

User Requirement

The user should be able to log in to their account.

Requirements vs. Specifications — Examples

Example 1: Online Shopping

User Requirement

The user should be able to place an order and pay online.

System Specifications

Accept credit card, debit card, and e-wallet payments

Send order confirmation to user's email after payment

Store order history for at least 12 months

Example 2: User Login

User Requirement

The user should be able to log in to their account.

System Specifications

Support login via email/password and social accounts

Lock account after 5 failed login attempts

Session must expire after 30 minutes of inactivity

Functional vs. Non-Functional Requirements

Functional Requirements

What the system **does**
Describes **actions** the user can perform

If missing → product **does not work**

E.g: *"User can reset password via email"*

E.g: *"User can place and track an order"*

Non-Functional Requirements

Constraints on how the system behaves

Not an action — a **quality or condition**

If missing → product works but **poorly**

E.g: *"System uptime must be 99.9%"*

E.g: *"App supports English & Vietnamese"*

Simple test:

Does removing it break a *feature*? → **Functional**.

Does it break a *quality*? → **Non-Functional**.

Non-Functional Requirement Categories

Product

Constraints on **how**
the product is built.

*"Must run on iOS &
Android"*

*"Supports 500+
users at once"*

Organizational

Company **policies &
standards.**

*"Code reviewed be-
fore merging"*

*"Features must have
unit tests"*

External

Outside **laws, regu-
lations or trends.**

*"No user data sold
(GDPR)"*

*"Must meet WCAG
accessibility"*

Example: Instagram

Functional

User can post a photo to their profile

Non-Functional

Photo must upload in under 3 seconds

3. WRPSM Model

WRSPM Model Introduction

A **reference model** connecting **requirements** & **specifications** to the real world.

W**World**

Assumptions
about the real
environment

R**Requirements**

What the user
wants

S**Specifications**

Technical
conditions to
meet

P**Program**

The code that
runs the system

M**Machine**

Hardware the
software runs on

WRPSM Model Introduction

A **reference model** connecting **requirements** & **specifications** to the real world.



World

Assumptions
about the real
environment



Requirements

What the user
wants



Specifications

Technical
conditions to
meet



Program

The code that
runs the system



Machine

Hardware the
software runs on

Why use this model?

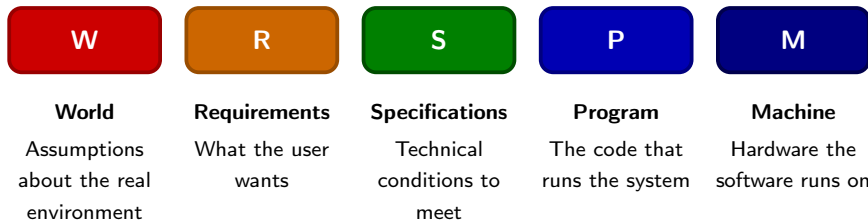
Organizes all ideas in one place

Easy to share with the whole team

Connects the user's world to the
final product

WRPSM Model Introduction

A **reference model** connecting **requirements** & **specifications** to the real world.



Why use this model?

- Organizes all ideas in one place
- Easy to share with the whole team
- Connects the user's world to the final product

Who reads each part?

- W, R → clients & managers
- S, P → developers
- M → hardware & infrastructure team

WRSPM — What Each Part Means

W — World Assumptions about the real environment.

Will there be Internet? What speed? Will there be power?

Ignoring these causes major failures — e.g. an ATM placed where the connection is too slow to process transactions.

R — Requirements What the **user** wants — simple, no tech terms.

"I want to withdraw money." / "I want to search on Google."

Just the goal — nothing about how it works.

S — Specifications Technical conditions that connect the requirement to the solution.

"The card reader contacts a back-end server to verify payment."

More technical, but still not the actual code.

WRSPM — What Each Part Means (cont.)

P — Program The actual **code** that implements everything above.

This is the software — it takes the specifications and turns them into something that runs. Text files, logic, ones and zeros.

M — Machine The **hardware** the software runs on.

Chips, buttons, power supply, money roller in an ATM...

Software cannot exist without hardware to run it. Every system needs physical components — from a simple button to a full server rack.

WRSPM — What Each Part Means (cont.)

P — Program The actual **code** that implements everything above.

This is the software — it takes the specifications and turns them into something that runs. Text files, logic, ones and zeros.

M — Machine The **hardware** the software runs on.

Chips, buttons, power supply, money roller in an ATM...

Software cannot exist without hardware to run it. Every system needs physical components — from a simple button to a full server rack.

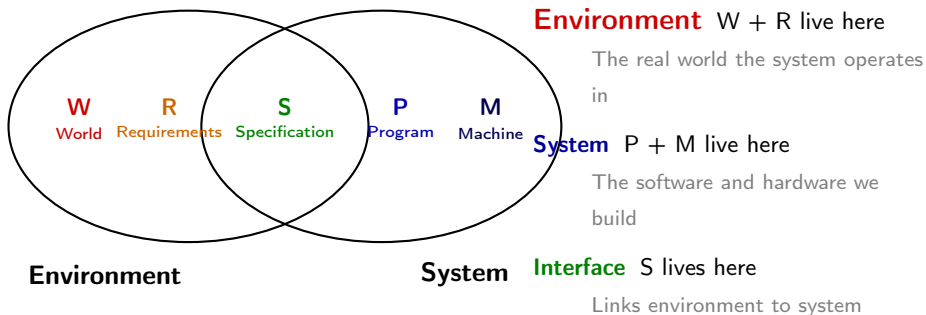
P without M?

Code with nowhere to run — useless.

M without P?

Hardware with no software — just a box.

WRPSM Visual Model



W, R $\xrightarrow{\text{defined by}}$ S $\xrightarrow{\text{implemented by}}$ P, M

WRSPM in Practice — ATM Example

WRSPM in Practice — ATM Example

W — World Where is the ATM placed? Is the location secure?

WRSPM in Practice — ATM Example

W — World Where is the ATM placed? Is the location secure?

Will there be electricity? Run off a generator? Who restocks it?

R — Requirements What does the user want?

"I want to withdraw money." — simple, no tech details.

S — Specifications How do we make that happen technically?

Card reader contacts back-end server. Encryption required. Timeout handling.

P — Program Code built to meet the specifications.

Engineers check specs continuously to ensure requirements are met.

M — Machine Hardware engineered to fit the specifications.

Cash roller, keypad, screen, network chip — all chosen based on specs.

If **P & M** meet **S** → **R** is satisfied → user gets what they need.

WRSPM Variables

There are 4 variables describing what is **visible** or **hidden** between the environment and the system:

Environment Side

eh — Environment Hidden Parts of the environment the system **cannot see**.

The system has no access to these, no matter what.

ev — Environment Visible Parts of the environment **exposed to the system**.

The interface through which environment talks to the system.

System Side

sv — System Visible Parts of the system **exposed to the environment**.

What the user can see and interact with.

sh — System Hidden Parts of the system **invisible to the environment**.

Internal workings the user never sees.

WRSPM Variables (cont.)

Environment Side

eh — **Environment Hidden** Parts of the environment the system **cannot see**.

The system has no access to these, no matter what.

ev — **Environment Visible** Parts of the environment **exposed to the system**.

The interface through which environment talks to the system.

System Side

sv — **System Visible** Parts of the system **exposed to the environment**.

What the user can see and interact with.

sh — **System Hidden** Parts of the system **invisible to the environment**.

Internal workings the user never sees.

These variables **change** depending on what the system is built to do. They are called *variables* because they vary per design.

WRSPM Variables - Examples

ATM Example

eh

WRSPM Variables - Examples

ATM Example

eh The physical card a user carries

WRSPM Variables - Examples

ATM Example

eh The physical card a user carries

The ATM has no idea a card exists
until it is inserted

ev

WRSPM Variables - Examples

ATM Example

- eh** The physical card a user carries
 - The ATM has no idea a card exists
 - until it is inserted
- ev** The mag strip / chip data

WRSPM Variables - Examples

ATM Example

eh The physical card a user carries

The ATM has no idea a card exists
until it is inserted

ev The mag strip / chip data

Once inserted, the system can read
its data

sv

WRSPM Variables - Examples

ATM Example

- eh** The physical card a user carries
 The ATM has no idea a card exists
 until it is inserted
- ev** The mag strip / chip data
 Once inserted, the system can read
 its data
- sv** Screen, buttons, UI

WRSPM Variables - Examples

ATM Example

eh The physical card a user carries

The ATM has no idea a card exists
until it is inserted

ev The mag strip / chip data

Once inserted, the system can read
its data

sv Screen, buttons, UI

What the user sees and interacts
with

sh

WRSPM Variables - Examples

ATM Example

- eh** The physical card a user carries
 The ATM has no idea a card exists
 until it is inserted
- ev** The mag strip / chip data
 Once inserted, the system can read
 its data
- sv** Screen, buttons, UI
 What the user sees and interacts
 with
- sh** Cash roller, back-end server calls

WRSPM Variables - Examples

ATM Example

eh The physical card a user carries
 The ATM has no idea a card exists
 until it is inserted

ev The mag strip / chip data
 Once inserted, the system can read
 its data

sv Screen, buttons, UI
 What the user sees and interacts
 with

sh Cash roller, back-end server calls
 Hidden inside — the user never sees
 this

WRSPM Variables - Examples

ATM Example

- eh** The physical card a user carries
 The ATM has no idea a card exists
 until it is inserted
- ev** The mag strip / chip data
 Once inserted, the system can read
 its data
- sv** Screen, buttons, UI
 What the user sees and interacts
 with
- sh** Cash roller, back-end server calls
 Hidden inside — the user never sees
 this

Heart Rate Monitor Example

eh

WRSPM Variables - Examples

ATM Example

- eh** The physical card a user carries
 The ATM has no idea a card exists
 until it is inserted
- ev** The mag strip / chip data
 Once inserted, the system can read
 its data
- sv** Screen, buttons, UI
 What the user sees and interacts
 with
- sh** Cash roller, back-end server calls
 Hidden inside — the user never sees
 this

Heart Rate Monitor Example

- eh** The heart beating itself

WRSPM Variables - Examples

ATM Example

eh The physical card a user carries
 The ATM has no idea a card exists
 until it is inserted

ev The mag strip / chip data
 Once inserted, the system can read
 its data

sv Screen, buttons, UI
 What the user sees and interacts
 with

sh Cash roller, back-end server calls
 Hidden inside — the user never sees
 this

Heart Rate Monitor Example

eh The heart beating itself
 No machine can directly observe the
 heart

ev

WRSPM Variables - Examples

ATM Example

- eh** The physical card a user carries
 The ATM has no idea a card exists
 until it is inserted
- ev** The mag strip / chip data
 Once inserted, the system can read
 its data
- sv** Screen, buttons, UI
 What the user sees and interacts
 with
- sh** Cash roller, back-end server calls
 Hidden inside — the user never sees
 this

Heart Rate Monitor Example

- eh** The heart beating itself
 No machine can directly observe the
 heart
- ev** Sound produced by each
 heartbeat

WRSPM Variables - Examples

ATM Example

eh The physical card a user carries
 The ATM has no idea a card exists
 until it is inserted

ev The mag strip / chip data
 Once inserted, the system can read
 its data

sv Screen, buttons, UI
 What the user sees and interacts
 with

sh Cash roller, back-end server calls
 Hidden inside — the user never sees
 this

Heart Rate Monitor Example

eh The heart beating itself
 No machine can directly observe the
 heart

ev Sound produced by each
 heartbeat
 The machine listens to this to
 measure heart rate

sv

WRPSM Variables - Examples

ATM Example

- eh** The physical card a user carries
 The ATM has no idea a card exists
 until it is inserted
- ev** The mag strip / chip data
 Once inserted, the system can read
 its data
- sv** Screen, buttons, UI
 What the user sees and interacts
 with
- sh** Cash roller, back-end server calls
 Hidden inside — the user never sees
 this

Heart Rate Monitor Example

- eh** The heart beating itself
 No machine can directly observe the
 heart
- ev** Sound produced by each
 heartbeat
 The machine listens to this to
 measure heart rate
- sv** Heart rate displayed on screen

WRPSM Variables - Examples

ATM Example

- eh** The physical card a user carries
 The ATM has no idea a card exists
 until it is inserted
- ev** The mag strip / chip data
 Once inserted, the system can read
 its data
- sv** Screen, buttons, UI
 What the user sees and interacts
 with
- sh** Cash roller, back-end server calls
 Hidden inside — the user never sees
 this

Heart Rate Monitor Example

- eh** The heart beating itself
 No machine can directly observe the
 heart
- ev** Sound produced by each
 heartbeat
 The machine listens to this to
 measure heart rate
- sv** Heart rate displayed on screen
 The reading the user sees

WRPSM Variables - Examples

ATM Example

- eh** The physical card a user carries
The ATM has no idea a card exists until it is inserted
- ev** The mag strip / chip data
Once inserted, the system can read its data
- sv** Screen, buttons, UI
What the user sees and interacts with
- sh** Cash roller, back-end server calls
Hidden inside — the user never sees this

Heart Rate Monitor Example

- eh** The heart beating itself
No machine can directly observe the heart
- ev** Sound produced by each heartbeat
The machine listens to this to measure heart rate
- sv** Heart rate displayed on screen
The reading the user sees
- sh** Signal processing inside the device

WRPSM Variables - Examples

ATM Example

- eh** The physical card a user carries
The ATM has no idea a card exists until it is inserted
- ev** The mag strip / chip data
Once inserted, the system can read its data
- sv** Screen, buttons, UI
What the user sees and interacts with
- sh** Cash roller, back-end server calls
Hidden inside — the user never sees this

Heart Rate Monitor Example

- eh** The heart beating itself
No machine can directly observe the heart
- ev** Sound produced by each heartbeat
The machine listens to this to measure heart rate
- sv** Heart rate displayed on screen
The reading the user sees
- sh** Signal processing inside the device
Hidden computation — user never sees this

WRSPM - World Example

Example: Vending Machine Software

Before writing any code, we must define the **world** this machine will operate in.



WRSPM - World Example

Example: Vending Machine Software

Before writing any code, we must define the **world** this machine will operate in.

Electricity is always available indoors

Internet connection is available

Placed in a controlled temperature environment

Users can read the on-screen instructions

Debit & credit cards are widely used

Store staff will refill products regularly



WRSPM - World Example

Example: Vending Machine Software

Before writing any code, we must define the **world** this machine will operate in.

Electricity is always available indoors

Internet connection is available

Placed in a controlled temperature environment

Users can read the on-screen instructions

Debit & credit cards are widely used

Store staff will refill products regularly

Key point: Change one assumption

→ the entire design may change.

e.g. No electricity → need battery pack.

Cash-based society → need coin slot.



WRSPM - Requirements Example

Example: Vending Machine Software

Requirements define **what** the system must do
— not *how*. Written in plain language.



WRSPM - Requirements Example

Example: Vending Machine Software

Requirements define **what** the system must do
— not *how*. Written in plain language.

User Requirements



WRSPM - Requirements Example

Example: Vending Machine Software

Requirements define **what** the system must do
— not *how*. Written in plain language.

User Requirements

Business Requirements

User can select a product

User can pay for the product

User can collect the
purchased product

User can change the display
language



WRSPM - Requirements Example

Example: Vending Machine Software

Requirements define **what** the system must do
— not *how*. Written in plain language.

User Requirements

- User can select a product
- User can pay for the product
- User can collect the purchased product
- User can change the display language

Business Requirements

- Machine can collect payment
- Machine can issue a receipt
- Machine can report low stock



WRSPM - Requirements Example

Example: Vending Machine Software

Requirements define **what** the system must do

— not *how*. Written in plain language.

User Requirements

User can select a product

User can pay for the product

User can collect the
purchased product

User can change the display
language

Business Requirements

Machine can collect
payment

Machine can issue a receipt

Machine can report low
stock

Key point: A missing requirement = a missing feature in the final product.

e.g. If "collect payment" is not listed, the machine may be built without it.



WRSPM - Specifications Example

Example: Vending Machine Software

Specifications **link** requirements to the solution. More technical
— but still no code.

From Requirements → Specifications



WRSPM - Specifications Example

Example: Vending Machine Software

Specifications **link** requirements to the solution. More technical — but still no code.

From Requirements → Specifications

User can pay → System uses chip & tap card reader + encrypted financial server connection

User can select product → Touchscreen display with product catalog interface

Machine can collect payment → Accepts debit/credit card via chip, tap, and mobile payment

User can change language → System supports Vietnamese, English, and French

Machine can report low stock → Sends alert to store staff when product count falls below threshold



WRSPM - Specifications Example

Example: Vending Machine Software

Specifications **link** requirements to the solution. More technical — but still no code.

From Requirements → Specifications

User can pay → System uses chip & tap card reader + encrypted financial server connection

User can select product → Touchscreen display with product catalog interface

Machine can collect payment → Accepts debit/credit card via chip, tap, and mobile payment

User can change language → System supports Vietnamese, English, and French

Machine can report low stock → Sends alert to store staff when product count falls below threshold

Key point: Specs answer "*How do we make that happen?*" — not yet choosing the exact technology, but narrowing it down.



WRSPM — The Full Picture

Vending Machine — from idea to buildable plan

W — World

- Electricity available indoors
- Internet available
- Debit/credit cards widely used
- Staff refill products regularly

R — Requirements

- User can select a product
- User can pay for the product
- Machine can collect payment
- User can change language

S — Specifications

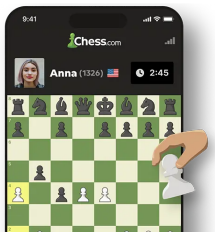
- Touchscreen product catalog
- Chip & tap card reader
- Encrypted server connection
- Multi-language UI support

P — Program The software built to meet the specs

M — Machine Hardware designed around the specs (card reader, screen, printer...)

3 hours with 4–5 people working through $W \rightarrow R \rightarrow S$
= a rough project proposal ready to hand to the engineering team.

Requirements in Practice — Chess App



The Problem

We have an existing **Chess app** (built in Java).

The client wants to **monetize** it.

Goal: Add a **subscription / pro** feature that generates revenue without rebuilding the app.

Approach: Use the **WRPSM** model to define $W \rightarrow R \rightarrow S$ before writing any code.

*Goal: cover **80%** of requirements — not every edge case, just enough to start building.*

Chess App — World & Requirements

W — World Assumptions

Chess App — World & Requirements

W — World Assumptions

- Smartphones support in-app payment
- Feasible monetization methods exist (ads, subscriptions)
- The existing codebase is extendable
- Users have access to an app store (Google Play)
- Internet connection is available on device

R — Requirements

Chess App — World & Requirements

W — World Assumptions

- Smartphones support in-app payment
- Feasible monetization methods exist (ads, subscriptions)
- The existing codebase is extendable
- Users have access to an app store (Google Play)
- Internet connection is available on device

R — Requirements

- User can subscribe to unlock pro features
- User can play without ads (pro only)
- User can cancel subscription within the app
- Pro features are removed when subscription expires
- Transaction must be completed in a timely fashion
- Feature must not add more than 5MB to app size

Note: Requirements come from both the **user** (ad-free experience) and the **business** (collect revenue, revoke on expiration).

Chess App — Specifications

S — Specifications (linking each requirement to a technical solution)

Language Feature coded in **Java** — existing app's language

Payment Use **Google Play Store** billing API for all transactions

Subscription check Check subscription status **daily** via Google's API
→ revoke pro features if invalid

Ads Disable **AdMob** once subscription is active
→ re-enable if subscription expires

Performance Transaction must complete within **500ms** after confirmation

Size Feature adds $\leq$ **5MB** to total app download size

With W + R + S defined, a developer receives a **clear direction** and knows exactly what questions remain to ask the client.